Expt No.10 Dt:

HEAT EXCHANGER PERFORMANCE ANALYSIS USING PANDAS AND SEABORN

Aim: To write and execute a Python program for analyzing experimental heat exchanger data and visualizing performance metrics using Pandas, Matplotlib, and Seaborn.

Interactive Development Environment (IDE) Used: Jupyter NoteBook
Programming Language Used : Python

Python Concepts used:

Python libraries are essential for engineering data analysis and visualization. Pandas provides tabular data structures for processing sensor data and calculating performance parameters. Matplotlib is a core library for creating static, 2D line and scatter plots. Seaborn builds on Matplotlib to provide high-level interfaces for attractive statistical graphics, such as bar charts and correlation heatmaps.

Python Concept	Example	Description
Pandas DataFrame	pd.DataFrame()	Stores tabular experimental data.
Matplotlib	plt.plot()	Creates line charts for temperature tracking.
Seaborn	sns.barplot()	Creates bar charts for comparative analysis.
Correlation Matrix	df.corr()	Calculates pairwise correlation of columns.
Heatmap	sns.heatmap()	Visualizes correlation data using color gradients.

Data Analysis Operation / Concept:

Data Analysis Operation / Concept	Example Code	Description
Import Visualization	import matplotlib.pyplot as plt	Imports the Matplotlib plotting module.
Import Seaborn	import seaborn as sns	Imports the Seaborn statistical plotting library.
Column Arithmetic	df["A"] - df["B"]	Performs element-wise calculations across columns.
Plot Bar Chart	sns.barplot(data=df, x=..., y=...)	Generates a bar plot mapped to DataFrame columns.
Plot Line Chart	plt.plot(x, y, marker="o")	Plots points connected by lines with custom markers.
Plot Heatmap	sns.heatmap(data.corr(), annot=True)	Generates a color-coded correlation matrix with value annotations.
Show Plot	plt.show()	Renders and displays the generated plots.

Python Program:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Step 1: Experimental heat exchanger data
data = {
   "Test": ["Test 1", "Test 2", "Test 3", "Test 4", "Test 5"],
   "Hot_Inlet_C": [80, 85, 90, 95, 100],
   "Hot_Outlet_C": [65, 68, 70, 73, 75],
   "Cold_Inlet_C": [25, 27, 28, 30, 32],
   "Cold_Outlet_C": [40, 43, 46, 49, 52]
```

```
}

df = pd.DataFrame(data)

# Step 2: Define operating parameters
mass_flow_rate = 0.1      # kg/s
specific_heat = 4180      # J/(kg degree C)

# Step 3: Calculate heat transfer rate on hot side
df["Heat_Transfer_kW"] = mass_flow_rate * specific_heat * (df["Hot_Inlet_C"] - df["Hot_Outlet_C"]) / 1000

# Step 4: Calculate heat capacity rates
hot_capacity = mass_flow_rate * specific_heat
cold_capacity = mass_flow_rate * specific_heat
# Maximum possible heat transfer
df["Q_Max_kW"] = min(hot_capacity, cold_capacity) * (df["Hot_Inlet_C"] - df["Cold_Inlet_C"]) / 1000

# Step 5: Calculate effectiveness
df["Effectiveness_pct"] = df["Heat_Transfer_kW"] / df["Q_Max_kW"] * 100

# Step 6: Display the results
print("HEAT EXCHANGER PERFORMANCE")
print(df.round(2).to_string(index=False))
print("\nAverage Heat Transfer Rate:", round(df["Heat_Transfer_kW"].mean(), 2), "kW")
print("Average Effectiveness:", round(df["Effectiveness_pct"].mean(), 2), "%")

# Step 7: Plot heat transfer rate
plt.figure(figsize=(7, 4))
sns.barplot(data=df, x="Test", y="Heat_Transfer_kW")
plt.title("Heat Transfer Rate for Each Test")
plt.xlabel("Experimental Test")
plt.ylabel("Heat Transfer Rate (kW)")
plt.tight_layout()
plt.show()

# Step 8: Plot hot and cold outlet temperatures
plt.figure(figsize=(7, 4))
plt.plot(df["Test"], df["Hot_Outlet_C"], marker="o", label="Hot outlet")
plt.plot(df["Test"], df["Cold_Outlet_C"], marker="s", label="Cold outlet")
plt.title("Outlet Temperature Comparison")
plt.xlabel("Experimental Test")
plt.ylabel("Temperature (°C)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# Step 9: Correlation analysis
columns = ["Hot_Inlet_C", "Hot_Outlet_C", "Cold_Inlet_C", "Cold_Outlet_C", "Heat_Transfer_kW",
"Effectiveness_pct"]
plt.figure(figsize=(9, 6))
sns.heatmap(df[columns].corr(), annot=True, cmap="coolwarm", vmin=-1, vmax=1)
plt.title("Heat Exchanger Parameter Correlations")
```

```
plt.tight_layout()
plt.show()
```

Sample Input and Output:
Input: The experimental test data are predefined in the Python dictionary in the program. No keyboard input is required during execution.

Test	Hot_Inlet_C	Hot_Outlet_C	Cold_Inlet_C	Cold_Outlet_C
Test 1	80	65	25	40
Test 2	85	68	27	43
Test 3	90	70	28	46
Test 4	95	73	30	49
Test 5	100	75	32	52

Expected Output:

Result: The Python program was written and executed successfully for analyzing experimental heat exchanger data. It successfully calculated heat transfer performance and generated accurate bar plots, line charts, and correlation heatmaps using Matplotlib and Seaborn.